Task 1. Create `deploy_playlist_app.sh`

This script deploys a simple Node.js Music Playlist API to an existing Ubuntu cloud server using SSH.

Prerequisites: An Ubuntu EC2 instance, Node.js and npm installed on the server, SSH access, and a project containing `app.js` and `package.json`.

`deploy_playlist_app.sh`

None

```
#!/usr/bin/env bash
set -euo pipefail

SERVER="${SERVER:?Set SERVER to the cloud server IP}"
SSH_USER="${SSH_USER:-ubuntu}"
APP_DIR="/opt/music-playlist"

echo "Deploying Music Playlist API..."

ssh "$SSH_USER@$SERVER" "
  sudo mkdir -p '$APP_DIR'
  sudo chown '$SSH_USER':'$SSH_USER' '$APP_DIR'
"

scp app.js package.json "$SSH_USER@$SERVER:$APP_DIR/"

ssh "$SSH_USER@$SERVER" "
  set -e
  cd '$APP_DIR'
  npm install --omit=dev

  if ! command -v pm2 >/dev/null 2>&1; then
    sudo npm install -g pm2
  fi

  pm2 start app.js --name music-playlist --update-env \
    || pm2 restart music-playlist --update-env
```

```
  pm2 save
"

echo "Deployment completed."
echo "Public URL: http://$SERVER:3000"
```

Run it from your local machine:

```
None
chmod +x deploy_playlist_app.sh

export SERVER="YOUR_EC2_PUBLIC_IP"
export SSH_USER="ubuntu"

./deploy_playlist_app.sh
```

The cloud server must already have Node.js, npm, SSH access, and an appropriate firewall rule. For a production deployment, configure HTTPS through a reverse proxy and manage application processes with a proper service manager or deployment platform.

Task 2. Deploy a Food Delivery Tracker using GitHub Actions and Heroku

Create a basic Express.js application and add a GitHub Actions workflow.

`package.json`

```
None
{
  "name": "food-delivery-tracker",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node app.js",
    "test": "node --test"
  },
  "dependencies": {
```

```
    "express": "^5.0.0"
  }
}
```

app.js

```
None
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Food Delivery Tracker is running!");
});

app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});

const port = process.env.PORT || 3000;

app.listen(port, "0.0.0.0", () => {
  console.log(`Server listening on ${port}`);
});
```

Create `.github/workflows/deploy.yml`:

```
None
name: Food Delivery Tracker Deployment

on:
  push:
    branches: [main]

permissions:
  contents: read
```

```yaml
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test

      - name: Deploy to Heroku
        uses: akhileshns/heroku-deploy@v3.14.15
        with:
          heroku_api_key: ${{ secrets.HEROKU_API_KEY }}
          heroku_app_name: ${{ secrets.HEROKU_APP_NAME }}
          heroku_email: ${{ secrets.HEROKU_EMAIL }}
```

Add these secrets under your GitHub repository's Settings → Secrets and variables → Actions:

- `HEROKU_API_KEY`
- `HEROKU_APP_NAME`
- `HEROKU_EMAIL`

Commit and push the application to `main`. The workflow runs tests first and attempts deployment only if they pass.

Note: Heroku's account, deployment, and pricing policies can change. Verify that your account can create and deploy an application before using this workflow. The third-party deployment action also needs to be reviewed and pinned to a verified commit SHA for production use.

Task 3. Run database migrations before starting a Movie Review app

For a Node.js application using a migration tool such as Knex, define the following scripts in `package.json`:

```
None
{
  "scripts": {
    "migrate": "knex migrate:latest",
    "start": "node app.js"
  }
}
```

Update the deployment sequence:

```
None
#!/usr/bin/env bash
set -euo pipefail

npm ci
npm run migrate
npm start
```

Explanation:

- `npm ci` installs dependencies from the lockfile.
- `npm run migrate` applies pending database migrations.
- `npm start` starts the server only after migrations succeed.
- `set -e` prevents the script from continuing after a failed command.

Testing procedure

1. Deploy to your configured test environment.
2. Check the deployment logs for the migration command.
3. Verify the migration records in the migration table used by your framework.
4. Confirm the application starts and can access the updated schema.

5. Test a deliberately invalid migration in a disposable environment to confirm that deployment stops on failure.

The actual migration command depends on the framework and database. Do not run untested schema changes against a production database.

Task 4. Secure environment variables for a Flipkart Wishlist microservice

Create a local `.env` file:

```
None
PORT=3000
DATABASE_URL=replace_with_database_connection
API_KEY=replace_with_api_key
```

Install the `dotenv` package:

```
None
npm install dotenv
```

Load the values in `app.js`:

```
None
require("dotenv").config();

const port = process.env.PORT || 3000;
const apiKey = process.env.API_KEY;
const databaseUrl = process.env.DATABASE_URL;

if (!apiKey || !databaseUrl) {
  throw new Error("Required environment variables are missing");
}

// Initialize the application and database using these values.
```

Add the following to `.gitignore`:

```
None
.env
node_modules/
```

For GitHub Actions, store secrets in the repository or environment settings and inject them only into the steps that need them.

```
None
- name: Deploy application
  env:
    API_KEY: ${{ secrets.WISHLIST_API_KEY }}
    DATABASE_URL: ${{ secrets.WISHLIST_DATABASE_URL }}
  run: npm run deploy
```

Here, `npm run deploy` represents your actual deployment command; define it in `package.json` for your target environment.

Security improvements:

- Never commit `.env` files or actual credentials.
- Grant the application only the database permissions it needs.
- Rotate exposed credentials immediately.
- Use a cloud secret manager for production deployments.
- Avoid printing secrets in logs.

Task 5. AI-generated deployment automation for a BookMyShow Event Reminder app

This example uses an existing Ubuntu server and deploys a simple Node.js reminder API.

`app.js`

```
None
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({
    app: "BookMyShow Event Reminder",
```

```
    status: "running"
  }));
});

server.listen(3000, "0.0.0.0");
```

package.json

```
None
{
  "name": "bookmyshow-event-reminder",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node app.js"
  }
}
```

deploy.sh

```
None
#!/usr/bin/env bash
set -euo pipefail

SERVER="${SERVER:?Set the server IP}"
SSH_USER="${SSH_USER:-ubuntu}"
APP_DIR="/opt/bookmyshow-reminder"

echo "Starting BookMyShow reminder deployment"

ssh "$SSH_USER@$SERVER" "
  sudo mkdir -p '$APP_DIR'
  sudo chown '$SSH_USER':'$SSH_USER' '$APP_DIR'
"
```

```
scp app.js package.json "$SSH_USER@$SERVER:$APP_DIR/"

ssh "$SSH_USER@$SERVER" "
  set -e
  cd '$APP_DIR'

  if ! command -v node >/dev/null 2>&1; then
    echo 'Node.js is not installed on the server'
    exit 1
  fi

  if ! command -v pm2 >/dev/null 2>&1; then
    sudo npm install -g pm2
  fi

  pm2 start app.js --name event-reminder --update-env \
    || pm2 restart event-reminder --update-env

  pm2 save
"

echo "Deployment finished"
echo "URL: http://$SERVER:3000"
```

Execute:

```
None
chmod +x deploy.sh

export SERVER="YOUR_EC2_PUBLIC_IP"
export SSH_USER="ubuntu"

./deploy.sh
```